

Solution to Problem 1 in Problem Sheet - Week 3

MATH70102 Unsupervised Learning

Dr Mikko Pakkanen, Department of Mathematics, Imperial College London

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import random
from sklearn.decomposition import KernelPCA, PCA

plt.style.use("ggplot")
```

Part (a)

Let us first load the data and print some of the top rows of the data matrix:

```
In [2]: simdata = np.loadtxt("simdata.csv", delimiter=",")
```

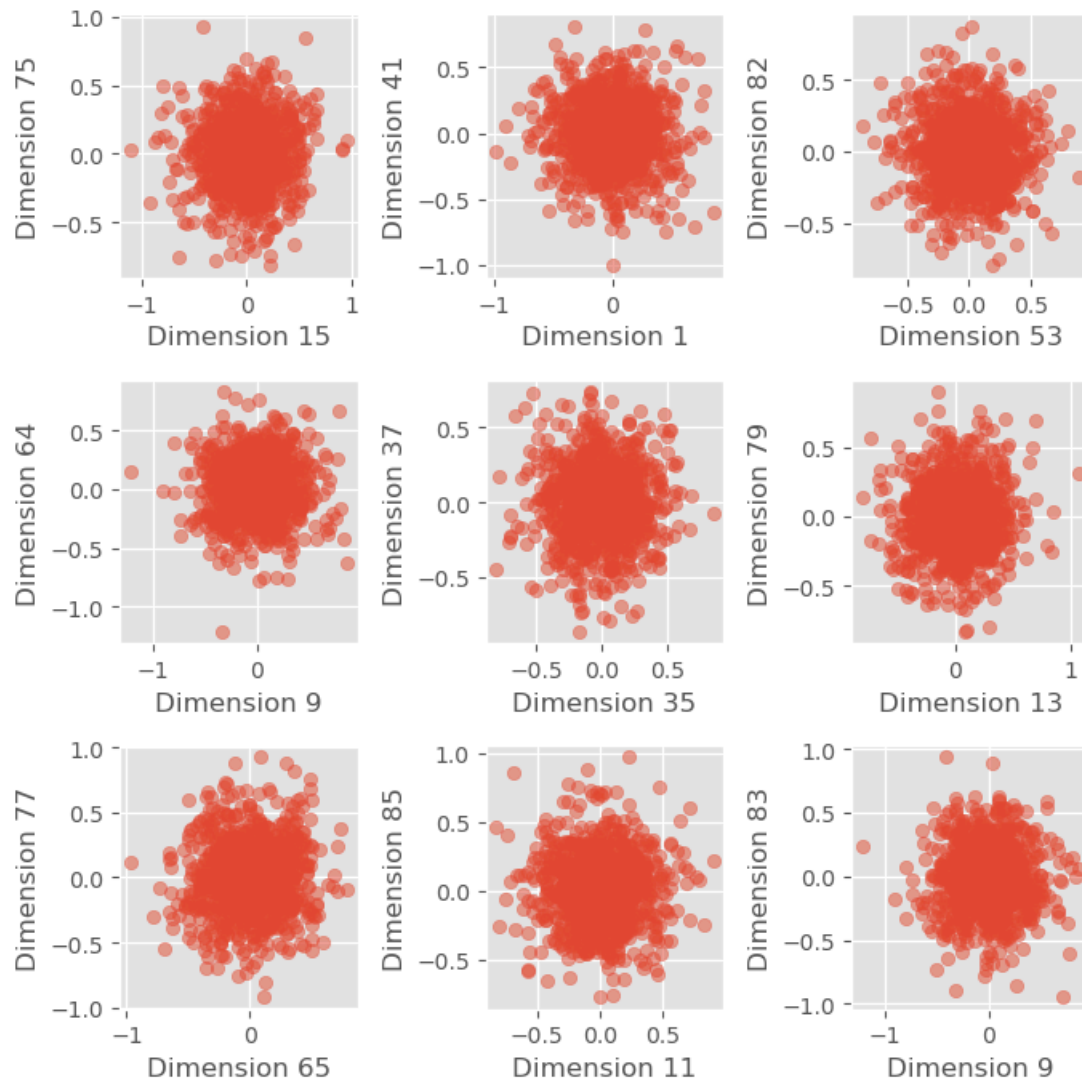
We will then randomly select a pair of dimensions and draw their scatterplot (doing it 9 times):

```
In [3]: pairs = random.sample(
    [(i, j) for i in range(simdata.shape[1]) for j in range(i + 1, simdata.shape[1])], 9
)

fig, axes = plt.subplots(3, 3, figsize=(7, 7))
axes = axes.ravel()

for ax, (i, j) in zip(axes, pairs):
    ax.scatter(simdata[:, i], simdata[:, j], alpha=0.5)
    ax.set_xlabel(f"Dimension {i}")
    ax.set_ylabel(f"Dimension {j}")

plt.tight_layout()
plt.show()
```



These do not reveal any obvious structure in the data.

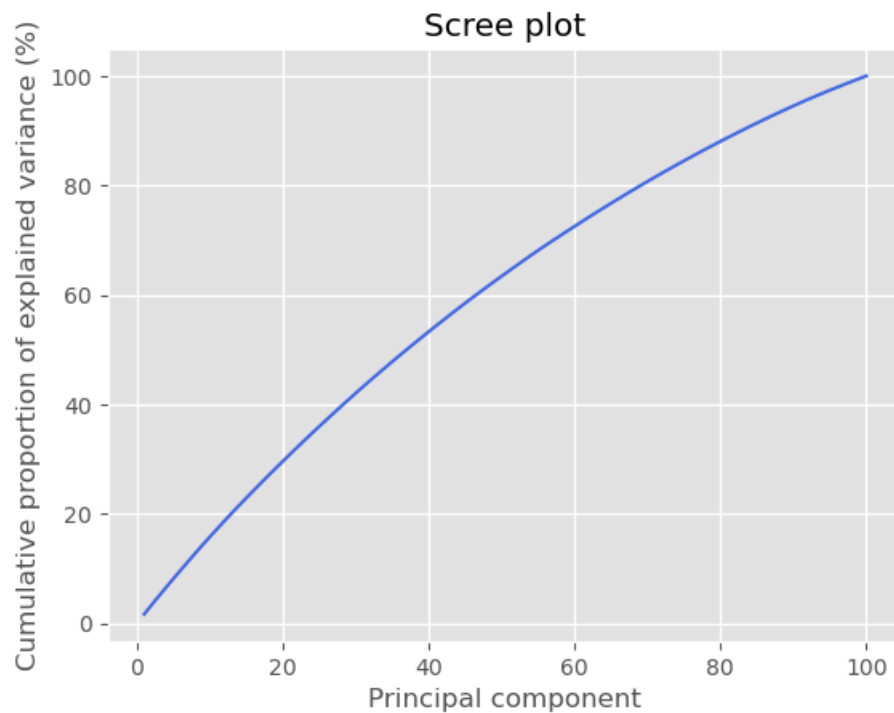
Part (b)

Let us apply PCA:

```
In [4]: simdata_pca = PCA()
simdata_pca.fit(simdata);
```

And draw a cumulative scree plot:

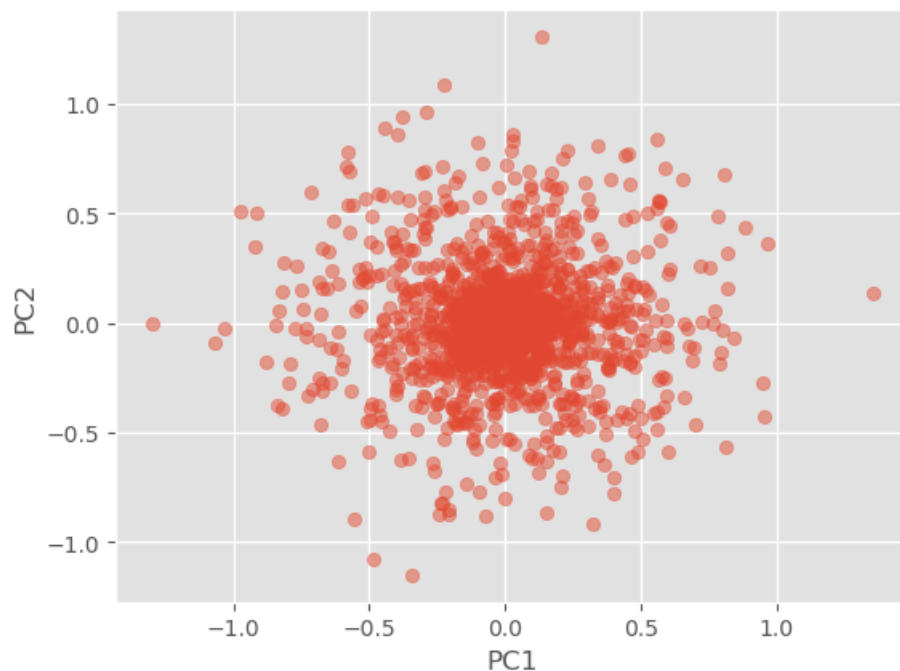
```
In [5]: plt.plot(
    range(1, 101),
    np.cumsum(simdata_pca.explained_variance_ratio_) * 100,
    color="royalblue",
)
plt.title("Scree plot")
plt.xlabel("Principal component")
plt.ylabel("Cumulative proportion of explained variance (%)")
plt.show()
```



Variance is rather evenly distributed across the principal components. Hence PCA may not be effective. Indeed, if we plot the first two principal components:

```
In [6]: simdata_pca = PCA(n_components=2)
simdata_pcs = simdata_pca.fit_transform(simdata)

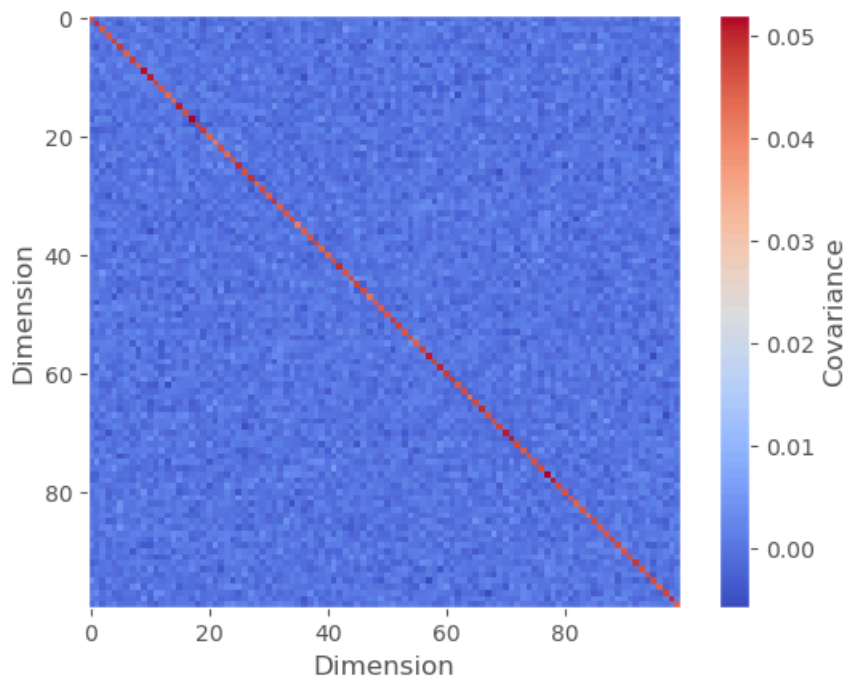
plt.scatter(simdata_pcs[:,0], simdata_pcs[:,1], alpha=0.5)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.show()
```



This does not convey any more structure than the scatterplots in part (a). But this would not be surprising if we knew that the sample covariance matrix of the data set is in fact **nearly diagonal**:

```
In [7]: simdata_cov = np.cov(simdata, rowvar=False)
plt.imshow(simdata_cov, cmap="coolwarm", interpolation="none")
plt.colorbar(label="Covariance")
plt.xlabel("Dimension")
plt.ylabel("Dimension")
```

```
plt.grid(False)
plt.show()
```



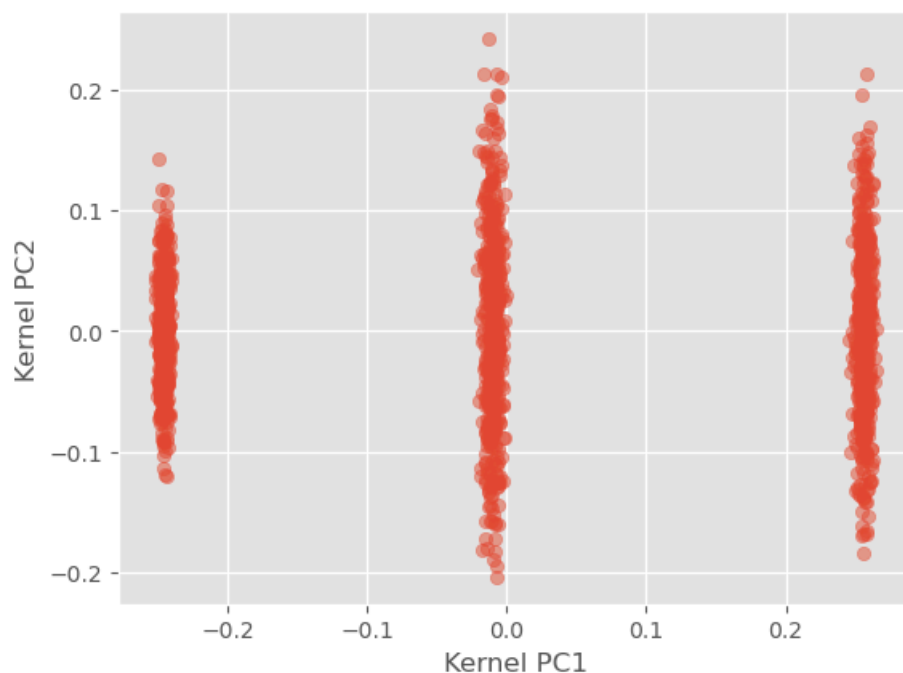
Part (c)

After the disappointing results from simple 2D scatterplots and PCA, we turn to kernel PCA. We perform kernel PCA using the radial basis function kernel with $\gamma = 0.1$, extracting the first two kernel PCs:

```
In [8]: simdata_kpca = KernelPCA(n_components=2, kernel="rbf", gamma=0.1)
simdata_kpcs = simdata_kpca.fit_transform(simdata)
```

Then we plot them:

```
In [9]: plt.scatter(simdata_kpcs[:,0], simdata_kpcs[:,1], alpha=0.5)
plt.xlabel("Kernel PC1")
plt.ylabel("Kernel PC2")
plt.show()
```



This reveals clear division into three groups. In fact, the data set was produced by sampling uniformly 500 points from each of the three spheres in $\mathbb{R}^{100}$:

$$\{\mathbf{x} \in \mathbb{R}^{100} : \|\mathbf{x}\| = 1\}$$

$$\{\mathbf{x} \in \mathbb{R}^{100} : \|\mathbf{x}\| = 2\}$$

$$\{\mathbf{x} \in \mathbb{R}^{100} : \|\mathbf{x}\| = 3\}$$

But this simple grouping is impossible to discern from the low-dimensional linear projections!